8.4. Analog-to-Digital Conversion

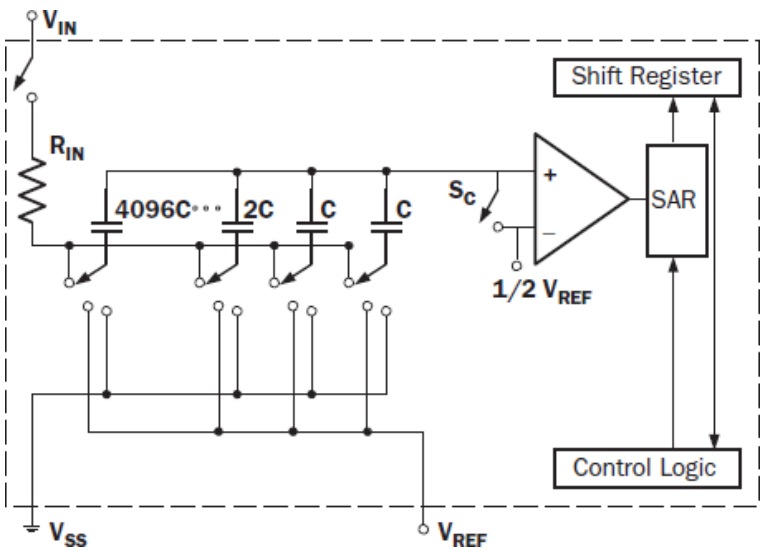
There are several analog-to-digital conversion methods each having its advantages and disadvantages [16]. The ADC14 module in MSP432 uses the successive approximation register (SAR) conversion method. Therefore, we will only deal with it in this section. Then, we will focus on the properties of the ADC14 module.

8.4.1. Successive Approximation Register Converter

As the name implies, the SAR converter works iteratively in obtaining the digital form of the analog signal. The MSB of the digital form is obtained first in iteration. Then, the remaining lower order bits are obtained step by step till the LSB.

The SAR circuitry is as in Fig. 8.3. In this figure, V_{IN} stands for the analog voltage value to be converted to digital form. The working principle of the given SAR circuitry is as follows. Initially, all the capacitors will be discharged to the offset voltage of the comparator. As the analog signal is fed to input, it will be kept at that value by a sample-and-hold circuit. Then, this voltage is applied to all capacitors. Since each capacitor has a different capacitance (in powers of two), they will be charged accordingly. These values are compared with the reference voltage. Based on the comparison, the bit value (either zero or one) is generated and saved in the shift register. Then, the reference voltage (V_{REF}) is updated and the conversion continues until the desired accuracy is obtained.

Figure 8.3 The circuit diagram of the SAR converter.



To explain how the SAR conversion works, we simulate it with the C code in Listing 8.7. Here, the constant `bitsize` represents the bitsize of the digital form to be obtained. The variable `Vin` stands for the analog voltage to be converted to digital form. The variable `Vref` stands for the reference voltage. The array `bits` holds the digital form obtained. Finally, the variable `quantized` holds the quantized approximate form of the analog input voltage. We can observe the `bits` and `quantized` variables from the Expressions window in CCS.

© McGraw-Hill Education. All rights reserved. Any use is subject to the Terms of Use, Privacy Notice and copyright information.

Listing 8.7 Simulation Program for the SAR Conversion

```
#include "msp.h"

#define bitsize 14

 float Vref = 3.6;
 float Vin = 3.3;
 float thresh;
 float quantized = 0;
 int count;
 int bitval;
 int bits[bitsize];

void main (void){

 WDT_A->CTL = WDT_A_CTL_PW | WDT_A_CTL_HOLD;

 Vref /= 2;
 thresh = Vref;

 for(count=0; count<bitsize; count++){
  Vref /= 2;
  if (Vin >= thresh){
   bitval = 1;
   thresh += Vref;}
  else {
   bitval = 0;
   thresh -= Vref;}

  bits[count] = bitval;
  quantized += 2*Vref*bitval;}

 while(1);
}
```

Let's give an example on the usage of this simulation program. Assume that we take the reference voltage (`Vref`) as 3 (V). We set the bitsize to 14 (bits). Assume the input voltage (`Vin`) to be 1.9 (V). We will get the digital representation `0x2888` as we run the simulation program. As can be observed from the Expressions window, the input voltage is approximated by 1.89990234 V. Hence, this simulation program gives insight on the working principles of the SAR conversion.

8.4.2. The ADC14 Module

The ADC14 module in MSP432 supports up to 1 MSPS 14-bit conversion. The result can be saved in 32 independent conversion registers controlled by 32 control registers. Therefore, each conversion register is controlled by its own control register. Using both conversion and control registers, 32 independent samples can be converted without any CPU intervention. The ADC14 module supports software selectable sample-and-hold-time, internal or external selectable reference voltage, selectable single and differential input channel, internal temperature sensor channel, selectable clock sources, and four (single channel, repeat single channel, sequence channel, and repeat sequence channel) conversion modes.

© McGraw-Hill Education. All rights reserved. Any use is subject to the Terms of Use, Privacy Notice and copyright information.

The ADC14 module uses two selectable voltage reference values (V_{R+} and V_{R-}) for conversion that define upper and lower limits. These reference voltages can be supplied by the REF_A module explained in Sec. 8.2 or by an external source. When the user decides on using an external source, he or she should make sure that the reference voltage is between 0 and 3.3 V. Otherwise, MSP432 may be damaged.

The ADC14 module can be used in either single channel or differential channel input settings. We will extensively use the first option throughout the book. For more information on the second option, please see [17].

For a single channel input (V_{in+}), the digital output value (N) obtained by the ADC14 module can be calculated as

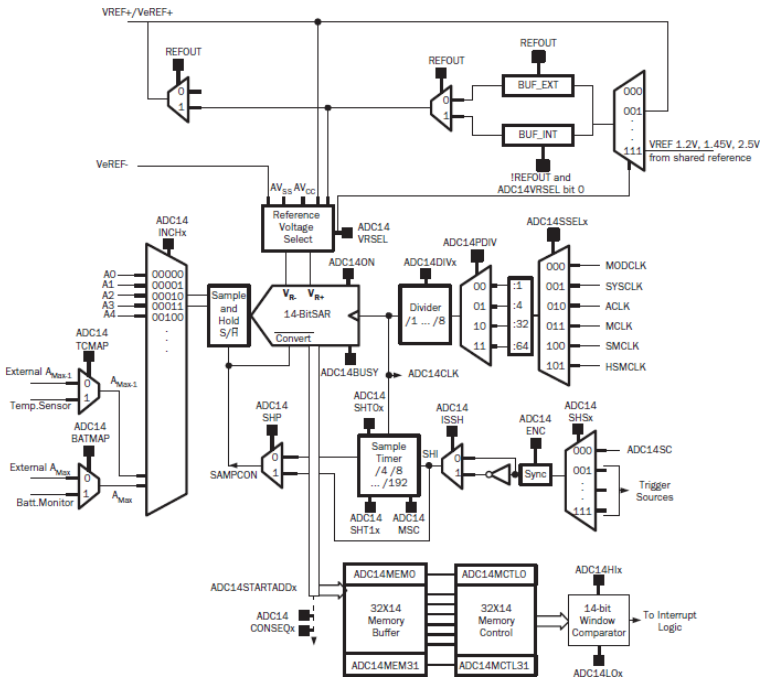
$$N = \frac{V_{in+} - V_{R-}}{V_{R+} - V_{R+}} \times 16384$$

(8.1)

where N is the converted digital value. The full scale value (`0x3FFF`) is obtained when V_{R+} or higher voltage is applied to the analog input. Similarly, zero value is obtained when V_{R-} or a lower voltage level is applied to the analog input.

A block diagram of the ADC14 module is as inFig. 8.4. As can be seen in this figure, the ADC14 module can sample various internal and external input channels using sample and hold circuit. The 14-bit ADC core can use the internal variable reference voltage generated by the REF_A module, external reference voltage, or supply voltage as its reference voltage. It can be triggered by various sources. The ADC core can be clocked by MODCLK, SYSCLK, ACLK, MCLK, SMCLK, and HSMCLK. The ADC14CLK clock can be generated by dividing these clocks. The conversion result will be stored in the configured conversion register(s) depending on the conversion mode selected.

Figure 8.4 Block diagram of the ADC14 module.



© McGraw-Hill Education. All rights reserved. Any use is subject to the Terms of Use, Privacy Notice and copyright information.

8.4.3. The Window Comparator

The ADC14 module has a window comparator that can be used to measure the input signal continuously. When the value of this signal falls between or out of predefined threshold levels, the corresponding interrupt will be generated. The user can set the low and high threshold levels. This gives high flexibility in applications.

Assume that we want to continuously check whether an input signal falls between two threshold levels. Without using the window comparator, the ADC14 module should be running continuously sampling the input signal. For each sampled signal, the CPU should be activated to check whether signal value falls between the two threshold levels. When the window comparator is used, the comparison operation is also performed within the module. Therefore, the CPU will not be activated till an interrupt is generated by the window comparator.

Let's consider an actual example where the usage of the window comparator simplifies operations. Assume that we want to process an audio signal in the environment whenever the signal level gets higher than a threshold. We can form a structure using the window comparator as follows. We can set the low threshold level such that when the audio signal value is below it, no ADC operation is done. When the audio signal value gets higher than the low threshold, then the window comparator triggers an interrupt. This interrupt starts the analog-to-digital conversion operation via the ADC14 module. Therefore, only the signal of interest is sampled via this operation. Then, the sampled signal can be processed. The main advantage of this setup is that the CPU will be in low-power mode for low-level signals. It will only be activated for the signals to be processed. Hence, the energy consumption of the system will be low compared to using the CPU all the times.

8.4.4. ADC14 Module Registers

There are 77 32-bit registers available to control the ADC14 module. These are ADC14CTL0, ADC14CTL1, ADC14LO0, ADC14HI0, ADC14LO1, ADC14HI1, ADC14MCTLx, ADC14MEMx, ADC14IER0, ADC14IER1, ADC14IFGR0, ADC14IFGR1, ADC14CLRIFGR0, ADC14CLRIFGR1, and ADC14IV where x can get values from 0 to 31. A structure named ADC14 is used to control these registers. Within this structure, the mentioned registers are named CTL0, CTL1, LO0, HI0, LO1, HI1, MCTL[x], MEM[x], IER0, IER1, IFGR0, IFGR1, CLRIFGR0, CLRIFGR1, and IV. Again, x can get values from 0 to 31.

© McGraw-Hill Education. All rights reserved. Any use is subject to the Terms of Use, Privacy Notice and copyright information.

The ADC14CTL0 register is used to configure and control the main properties of the ADC14 module. Entries of this register are tabulated in Table 8.9. Here, ADC14PDIV bits are used to select the predivider for the ADC14 clock. Constants for these bits are ADC14_CTL0_PDIV_x, where x can get values from 0 to 3. Corresponding predivider values are 1, 4, 32, and 64, respectively. ADC14SHSx bits are used for selecting sample-and-hold source for the ADC14 module. Constants for these bits are ADC14_CTL0_SHS_x, where x can get values from 0 to 7. Corresponding sample-and-hold sources are ADC14SC bit, Timer_A0 output 1, Timer_A0 output 2, Timer_A1 output 1, Timer_A1 output 2, Timer_A2 output 1, Timer_A2 output 2, and Timer_A3 output 1, respectively. The ADC14SHP bit is used to select the source of the sampling signal. When this bit is set with the constant ADC14_CTL0_SHP, the sampling signal is sourced from the output of the sampling timer. When the bit is reset, the sampling signal is sourced from the input sample signal directly. The ADC14ISSH bit is used to invert the sample input signal. When this bit is set, with the constant ADC14_CTL0_ISSH, the sample input signal is inverted. ADC14DIVx bits are used to select the ADC14 clock divider. Constants for these bits are ADC14_CTL0_DIV_x, where x can get values from 0 to 7. Corresponding divider values are 1, 2, 3, 4, 5, 6, 7, and 8, respectively. ADC14SSELx bits are used to select the clock for the ADC14 module. Constants for these bits are ADC14_CTL0_SSEL_x, where x can get values from 0 to 5. Corresponding clocks are MODCLK, SYSCLK, ACLK, MCLK, SMCLK, and HSMCLK, respectively. ADC14CON-SEQx bits are used to select the conversion mode. Constants for these bits are ADC14_CTL0_CONSEQ_x, where x can get values from 0 to 3. Corresponding conversion modes are single-channel single-conversion, multi-channel single-conversion, single-channel repeated-conversion, and multi-channel repeated-conversion, respectively. The ADC14BUSY bit is used to observe the ADC operation activity. When this bit is set by the ADC14 module, it indicates that an active sampling or conversion process is going on. Constant for this bit is ADC14_CTL0_BUSY. ADC14SHT1x bits are used to define sample-and-hold time in terms of ADC14 clock cycles for the ADC14MEMx registers from 8 to 23. Constants for these bits are ADC14_CTL0_SHT1_x, where x can get values from 0 to 7. Corresponding ADC14 clock cycle values are calculated by the formula 4×2^x. ADC14SHT0x bits are used to define sample-and-hold time in terms of ADC14 clock cycles for the ADC14MEMx registers from 0 to 7 and 24 to 31. Constants for these bits are ADC14_CTL0_SHT0_x, where x can get values from 0 to 7. Corresponding ADC14 clock cycle values are calculated by 4×2^x. The ADC14MSC bit allows multiple sample and conversion operations for valid sampling modes. When this bit is set, with the constant ADC14_CTL0_MSC, the sampling and conversion operations after the first one are performed automatically. The ADC14ON bit is used to enable the ADC14 module. Constant for this bit is ADC14_CTL0_ON. The ADC14ENC bit enables the conversion. Constant for this bit is ADC14_CTL0_ENC. The ADC14SC bit starts the analog-to-digital conversion. Constant for this bit is ADC14_CTL0_SC.

Table 8.9 ADC14 Module Control Register 0 (ADC14CTL0)

Bits	31-10	29-27	26	25	24-22	21-19	18-17
	ADC14PDIV	ADC14SHSx	ADC14SHP	ADC14ISSH	ADC14DIVx	ADC14SSELx	ADC14CONSEQx
Bits	16	15-12	11-8	7	4	1	0
	ADC14BUSY	ADC14SHT1x	ADC14SHT0x	ADC14MSC	ADC14ON	ADC14ENC	ADC14SC

© McGraw-Hill Education. All rights reserved. Any use is subject to the Terms of Use, Privacy Notice and copyright information.

The ADC14CTL1 register is mainly used to control the resolution and power consumption of the ADC14 module. Also, some special input channels can be activated through this register. Entries of the ADC14CTL1 register are tabulated in Table 8.10. The ADC14TCMAP bit should be used to enable the internal temperature sensor for the ADC14 \nobreak module. When this bit is set, with the constant ADC14_CTL1_TCMAP, the internal temperature sensor channel is enabled and can be used via channel A22. The ADC14BATMAP bit is used to monitor the voltage level of the analog power supply (AV_{CC}) of the microcontroller. It is especially needed when the microcontroller is supplied by external battery. When this bit is set, with constant ADC14_CTL1_BATMAP, internal 1/2xAV_{CC} channel is enabled and can be used via channel A23. Half of theAV_{CC} voltage can be obtained from this channel. ADC14RES bits are used to select the resolution of the ADC14 module. Constants for these bits are ADC14_CTL1_RES_x, where x can get values from 0 to 3. Corresponding resolution values are 8, 10, 12, and 14 bits, respectively. The ADC14REFBURST bit is used to select on-time of reference buffer. When this bit is set, with the constant ADC14_CTL1_REFBURST, the ADC14 reference buffer is only enabled during the sample and conversion process. When this bit is reset, the buffer will be enabled continuously. ADC14PWRMD bits are used to select power modes for the ADC14 module. Constants for these bits are ADC14_CTL1_PWRMD_x, where x can get values 0 or 2. When ADC14_CTL1_PWRMD_0 is selected, the ADC14 module is used in regular power mode with up to 1 MSPS sample rate. Also, any resolution can be used in this mode. When ADC14_CTL1_PWRMD_2 is selected, the ADC14 module is used in low-power mode with up to 200 kSPS sample rate. Only 8-, 10-, or 12-bit resolution can be used in this mode. When this mode is used, power consumption of the ADC14 module is less than half of the regular mode. For further information on the remaining bits of this register, please see [4].

Table 8.10 ADC14 Module Control Register 1 (ADC14CTL1)

Bits	23	22	5-4	2	1-0
	ADC14TCMAP	ADC14BATMAP	ADC14RES	ADC14REFBURST	ADC14PWRMD

ADC14LO0 and ADC14LO1 registers are used to set low threshold value for window comparator. On the other hand, ADC14HI0 and ADC14HI1 registers are used to set high threshold value for the window comparator.

The ADC14MCTLx register is mainly used to select ADC references and input channels. It also controls the window comparator. Entries of this register are tabulated in Table 8.11. Here, ADC14WINCTH is used to select window comparator registers 0 or 1. Constant for this bit is ADC14_MCTLN_WINCTH. If this bit is reset, ADC14LO0 and ADC14HI0 registers are used. If it is set, ADC14LO1 and ADC14HI1 registers are used. ADC14WINC bit is used to enable window comparator. Constant for this bit is ADC14_MCTLN_WINC. ADC14VRSEL bits are used to select the voltage reference values. Constants for these bits are ADC14_MCTLN_VRSEL_x, where x can get values 0, 1, 14, and 15. Corresponding reference values are given in Table 8.12. Here, AV_{CC} is the analog power supply of the ADC14 module with the nominal value 3 V.AV_{SS} is the analog ground supply voltage of the ADC14 module with the nominal value 0 V. V_{REF} is the internal positive reference voltage that is generated by the REF_A module and shared with other modules. V_{eREF+} is the external positive reference voltage for the ADC14 module that can be obtained from pin P5.6. V_{eREF-} is the external negative reference voltage for the ADC14 module that can be obtained from pin P5.7. It is advised to connect V_{eREF-} to AV_{SS}.

Table 8.11 ADC14 Module Conversion Memory Control Register x (ADC14MCTLx)

Bits	15	14	11-8	7	4-0
	ADC14WINCTH	ADC14WINC	ADC14VRSEL	ADC14EOS	ADC14INCHx

© McGraw-Hill Education. All rights reserved. Any use is subject to the Terms of Use, Privacy Notice and copyright information.

Table 8.12 Reference Values Used in the ADC14 Module

Constant	V_{R+}	V_{R-}
ADC14_MCTLN_VRSEL_0	AV_{CC}	AV_{SS}
ADC14_MCTLN_VRSEL_1	V_{REF} buffered	AV_{SS}
ADC14_MCTLN_VRSEL_14	V_{eREF+}	V_{eREF-}
ADC14_MCTLN_VRSEL_15	V_{eREF+} buffered	V_{eREF-}

The ADC14EOS bit is used to stop conversion in multi-channel conversion mode. When this bit is set, with the constant ADC14_MCTLN_EOS, related input channel is specified as the last input channel to be converted. ADC14INCHx bits are used for selecting the input channels for the ADC14 module. Constants for these bits are ADC14_MCTLN_INCH_x, where x can get values from 0 to 31 to select the input channel from A0 to A31, respectively. The pin usage of the ADC14 module is as in Table 8.13. For further information on the remaining bits of the ADC14MCTLx register, please see [4].

Table 8.13 Pin Usage of the ADC14 Module

Pin	Usage Area
P5.5	ADC14 analog input A0
P5.4	ADC14 analog input A1
P5.3	ADC14 analog input A2
P5.2	ADC14 analog input A3
P5.1	ADC14 analog input A4
P5.0	ADC14 analog input A5
P4.7	ADC14 analog input A6
P4.6	ADC14 analog input A7
P4.5	ADC14 analog input A8
P4.4	ADC14 analog input A9
P4.3	ADC14 analog input A10
P4.2	ADC14 analog input A11
P4.1	ADC14 analog input A12
P4.0	ADC14 analog input A13
P6.1	ADC14 analog input A14
P6.0	ADC14 analog input A15

© McGraw-Hill Education. All rights reserved. Any use is subject to the Terms of Use, Privacy Notice and copyright information.

Pin	Usage Area
P9.1	ADC14 analog input A16
P9.0	ADC14 analog input A17
P8.7	ADC14 analog input A18
P8.6	ADC14 analog input A19
P8.5	ADC14 analog input A20
P8.4	ADC14 analog input A21
P8.3	ADC14 analog input A22
P8.2	ADC14 analog input A23
P5.6	ADC14 positive reference voltage V_{REF+}/V_{eREF+}
P5.7	ADC14 negative reference voltage V_{REF-}/V_{eREF-}

ADC14MEMx registers are used for keeping the conversion results, where x can get values from 0 to 31. This means that there are 32 different 32-bit memory locations to keep conversion results. For single-channel conversion, the result is written to the ADC14MEM0 register. For multi-channel conversion, the first result is written to the ADC14MEM0 register, then other ADC14MEMx registers are used for the remaining channels.

The ADC14IER0 register is used to enable the ADC interrupt for the selected ADC14MEMx register. Constants for this register are ADC14_IER0_IEx, where x can get values from 0 to 31. These constants enable interrupts for the ADC14MEMx register from 0 to 31, respectively. An interrupt is generated when a conversion result is written to the ADC14MEMx register. To note here, the interrupt should have been enabled for that ADC14MEMx register.

The ADC14IER1 register is used to enable the ADC interrupt for the window comparator. Entries of this register are tabulated in Table 8.14. Here, ADC14HIIE is used to enable the interrupt for exceeding the high threshold of window comparator. Constant for this bit is ADC14_IER1_HIIE. ADC14LOIE is used to enable the interrupt for falling below the low threshold of window comparator. Constant for this bit is ADC14_IER1_LOIE. ADC14INIE is used to enable the interrupt for staying between the thresholds of window comparator. Constant for this bit is ADC14_IER1_INIE. For further information on the remaining bits of the ADC14IER1 register, please see [4].

Table 8.14 ADC14 Module Interrupt Enable Register 1 (ADC14IER1)

Bits	3	2	1
	ADC14HIIE	ADC14LOIE	ADC14INIE

The ADC14IFGR0 register is the interrupt flag for the ADC14 interrupt. Constants for this register are ADC14_IFGR0_IFGx, where x can get values from 0 to 31. When a conversion result is written to an ADC14MEMx register, related interrupt flag is set.

© McGraw-Hill Education. All rights reserved. Any use is subject to the Terms of Use, Privacy Notice and copyright information.

The ADC14IFGR1 register is the interrupt flag for the window comparator. Entries of this register are tabulated in Table 8.15. Here, ADC14HIIFG is the interrupt flag for exceeding the high threshold of window comparator. Constant for this bit is ADC14_IFGR1_HIIFG. ADC14LOIE is the interrupt flag for falling below the low threshold of window comparator. Constant for this bit is ADC14_IFGR1_LOIFG. ADC14INIE is the interrupt flag for staying between the thresholds of window comparator. Constant for this bit is ADC14_IFGR1_INIFG. For further information on the remaining bits of the ADC14IFGR1 register, please see [4].

Table 8.15 ADC14 Module Interrupt Flag Register 1 (ADC14IFGR1)

Bits	3	2	1
	ADC14HIIFG	ADC14LOIFG	ADC14INIFG

The ADC14CLRIFGR0 register is used for clearing the selected interrupt flags. Constants for this register are ADC14_CLRIFGR0_CLRIFGx, where x can get values from 0 to 31. These constants clear interrupt flags from 0 to 31, respectively.

The ADC14CLRIFGR1 register is used for clearing the interrupt flags for window comparator. Entries of this register are tabulated in Table 8.16. Here, the CLRADC14HIIFG is used to clear the ADC14HIIFG (with the constant ADC14_CLRIFGR1_CLRHIIFG). The CLRADC14LOIFG is used to clear the ADC14LOIFG (with the constant ADC14_CLRIFGR1_CLRLOIFG). The CLRADC14INIFG is used to clear the ADC14INIFG (with the constant ADC14_CLRIFGR1_CLRINIFG). For further information on the remaining bits of the ADC14CLRIFGR1 register, please see [4].

Table 8.16 ADC14 Module Clear Interrupt Flag Register 1 (ADC14CLRIFGR1)

Bits	3	2	1
	CLRADC14HIIFG	CLRADC14LOIFG	CLRADC14INIFG

The ADC14IV register is the interrupt vector used for all ADC14 interrupts. The contents of this register are given in Table 8.17.

© McGraw-Hill Education. All rights reserved. Any use is subject to the Terms of Use, Privacy Notice and copyright information.

Table 8.17 ADC14IV Register

TAIV Content	Interrupt Source	Interrupt Flag	Interrupt Priority
0x00	No interrupt pending	-	-
0x02	ADC14MEMx overflow	ADC14OVIFG	Highest
0x04	Conversion time overflow	ADC14TOVIFG	
0x06	ADC14 window high interrupt flag	ADC14HIIFG	
0x08	ADC14 window low interrupt flag	ADC14LOIFG	
0x0A	ADC14 window between interrupt flag	ADC14INIFG	
0x0C	ADC14MEM0 interrupt flag	ADC14IFG0	
0x0E	ADC14MEM1 interrupt flag	ADC14IFG1	
...	...	...	
0x48	ADC14MEM30 interrupt flag	ADC14IFG30	
0x4A	ADC14MEM31 interrupt flag	ADC14IFG31	
0x4C	Local buffered reference is ready	ADC14RDYIFG	Lowest

8.4.5. ADC14 Module Usage via DriverLib Functions

The ADC14 module can also be controlled via predefined DriverLib functions. We provide functions of interest below. The complete function list and macro definitions can be found in [11].

```
void ADC14_enableModule(void)
/*
Enables the ADC14 module by modifying the ADC14ON bit of the ADC14CTL0
    register.
*/

bool ADC14_disableModule(void)
/*
Disables the ADC14 module by modifying the ADC14ON bit of the ADC14CTL0
     register.
```

© McGraw-Hill Education. All rights reserved. Any use is subject to the Terms of Use, Privacy Notice and copyright information.

```
*/

bool ADC14_initModule(uint32_t clockSource, uint32_t clockPredivider,
    uint32_t clockDivider, uint32_t internalChannelMask)
/*
Configures and initializes the ADC14 module by modifying the ADC14CTL0
    and ADC14CTL1 registers.
clockSource: clock to feed the ADC14 module. It can be one of MODCLK,
    SYSCLK, ACLK, MCLK, SMCLK, and HSMCLK. Please see DriverLib user
    guide for macro definitions.
clockPredivider:  pre-divider for the clock. It can be one of 1, 4, 32,
     or 64. Please see DriverLib user guide for macro definitions.
clockDivider: clock divider for the pre-divided clock. It can be one of
     1, 2, 3, 4, 5, 6, 7, or 8. Please see DriverLib user guide for
    macro definitions.
internalChannelMask: the channel mapping for the ADC14 module. It can
    be logical OR form of the internal channel 0, 1, 2, 3, temperature
    sensor, and battery channels. No internal channel routing can be
    selected if desired. Please see DriverLib user guide for macro
    definitions.
*/

void ADC14_setResolution(uint32_t resolution)
/*
Sets the sampling resolution of the ADC14 module by modifying ADC14RES
    bits of the ADC14CTL1 register.
resolution: resolution to be set. It can be one of ADC_8BIT, ADC_10BIT,
     ADC_12BIT or ADC_14BIT.
*/

bool ADC14_setSampleHoldTrigger(uint32_t source, bool invertSignal)
/*
Sets the trigger for the ADC14 module by modifying the ADC14SHSx bits
    of the ADC14CTL0 register.
source: the trigger source. It can be one of software, 1, 2, 3, 4, 5,
    6, or 7 values. Please see DriverLib user guide for macro
    definitions.
invertSignal: when set to logic level 1, the source triggers the ADC at
     falling edge.
*/

bool ADC14_setSampleHoldTime(uint32_t firstPulseWidth, uint32_t
    secondPulseWidth)
/*
Sets the sample-and-hold-time by modifying the ADC14SHT1x or ADC14SHT0x
     bits of ADC14CTL0 register. There are two configuration parameters
     to control the sample-and-hold-time for different memory
     locations.
firstPulseWidth: first sample and hold duration parameter for memory
    locations ADC_MEMORY_0 through ADC_MEMORY_7 and ADC_MEMORY_24
    through ADC_MEMORY_31 in terms of ADC clock cycles. It can be one
    of 4, 8, 16, 32, 64, 96, 128 or 192. Please see DriverLib user
    guide for macro definitions.
```

© McGraw-Hill Education. All rights reserved. Any use is subject to the Terms of Use, Privacy Notice and copyright information.

```
secondPulseWidth: second sample and hold duration parameter for memory
    locations ADC_MEMORY_8 through ADC_MEMORY_23 in terms of ADC clock
    cycles. Possible values are the same as firstPulseWidth. Please see
     DriverLib user guide for macro definitions.
*/

bool ADC14_configureMultiSequenceMode(uint32_t memoryStart, uint32_t
    memoryEnd, bool repeatMode)
/*
Configures the ADC14 module in multi channel (scan) mode by modifying
    the ADC14CTL0 and ADC14CTL1 registers. The results are saved into
    ADC memory locations.
memoryStart: start address of the memory locations. It can be one of
    ADC_MEM0 through ADC_MEM31.
memoryEnd: end address of the memory locations. It can be one of
    ADC_MEM0 through ADC_MEM31.
repeatMode: configures the ADC14 module to repeat conversion
    automatically after first conversion. It can be at logic level 1
    or 0.
*/

bool ADC14_configureSingleSampleMode(uint32_t memoryDestination, bool
    repeatMode)
/*
Configures the ADC14 module in single channel mode by modifying
    ADC14CTL0 and ADC14CTL1 registers. The result is saved in ADC
    memory location.
memoryDestination: the memory location to save the result. It can be
    one of ADC_MEM0 through ADC_MEM31.
repeatMode: configures the ADC14 module to repeat conversion
    automatically after the first conversion. It can be at logic level
    1 or 0.
*/

bool ADC14_enableConversion(void)
/*
Enables the configured conversion mode by modifying the ADC14ENC bit of
     the ADC14CTL0 register. Note that it does not trigger the
    conversion.
*/

void ADC14_disableConversion(void)
/*
Stops and disables the configured conversion mode by modifying the
    ADC14ENC bit of the ADC14CTL0 register.
*/

bool ADC14_isBusy(void)
/*
Returns logic level 1 if the ADC14 module is active and conversion is
    in progress by checking the ADC14BUSY bit of the ADC14CTL0
    register.
*/
```

© McGraw-Hill Education. All rights reserved. Any use is subject to the Terms of Use, Privacy Notice and copyright information.

```
bool ADC14_configureConversionMemory(uint32_t memorySelect, uint32_t
    refSelect, uint32_t channelSelect, bool differntialMode)
/*
Configures the conversion memory locations by individually or in group
    by modifying ADC14MCTLx registers.
memorySelect: memory location to be configured. It can be logical OR
    form of ADC_MEM0 through ADC_MEM31.
refSelect: voltage reference for conversion. It can be one of power
    supply voltages, internal or external reference voltages. Please
    see DriverLib user guide for macro definitions.
channelSelect: the channel to be sampled. It can be one of ADC_INPUT_A0
     through ADC_INPUT_A31.
differntialMode: sets the conversion in single or differential mode. It
     can be one of ADC_NONDIFFERENTIAL_INPUTS or
    ADC_DIFFERENTIAL_INPUTS. If differential mode is enabled, then the
    configured channel is combined with its neighbor channel to set a
    differential channel. For example if A0 or A1 channel is
    configured, then conversation takes place between A0 and A1
        channels.
*/

bool ADC14_enableComparatorWindow(uint32_t memorySelect, uint32_t
    windowSelect)
/*
Enables the window comparator by modifying the ADC14MCTLx register.
memorySelect: the selected memory location. It can be logical OR form
    of ADC_MEM0 through ADC_MEM31.
windowSelect: memory location to store the converted samples. It can be
     one of ADC_COMP_WINDOW0 or ADC_COMP_WINDOW1 values.
*/

bool ADC14_disableComparatorWindow(uint32_t memorySelect)
/*
Disables the window comparator by modifying the ADC14MCTLx register.
memorySelect: the selected memory location. It can be logical OR form
    of ADC_MEM0 through ADC_MEM31.
*/

bool ADC14_setComparatorWindowValue(uint32_t window, int16_t low,
    int16_t high)
/*
Sets the lower and upper threshold levels of window comparator by
    modifying ADC14LOx and ADC14HIx registers.
window: memory location to store the converted samples. It can be one
    of ADC_COMP_WINDOW0 or ADC_COMP_WINDOW1 values.
low: lower threshold value.
high: upper threshold value.
*/

uint_fast16_t ADC14_getResult(uint32_t memorySelect)
/*
Returns the conversion result from the selected memory location by
    reading the contents of individual ADC14MEMx registers.
memorySelect: the selected memory location. It can be one of ADC_MEM0
    through ADC_MEM31.
```

© McGraw-Hill Education. All rights reserved. Any use is subject to the Terms of Use, Privacy Notice and copyright information.

```
*/

void ADC14_getMultiSequenceResult(uint16_t *res)
/*
Returns the conversion results from configured multi-sequence memory
    addresses by reading the contents of ADC14MEMx registers.
res: the array pointer to save conversion results.
*/

void ADC14_getResultArray(uint32_t memoryStart, uint32_t memoryEnd,
    uint16_t *res)
/*
Returns the conversion results from the selected memory addresses by
    reading the contents of ADC14MEMx registers.
memoryStart: start address of the memory locations. It can be one of
    ADC_MEM0 through ADC_MEM31.
memoryEnd: end address of the memory locations. It can be one of
    ADC_MEM0 through ADC_MEM31.
res: the array pointer to save conversion results.
*/

bool ADC14_enableSampleTimer(uint32_t multiSampleConvert)
/*
Enables the timer trigger for sampling by modifying ADC14SHP and
    ADC14MSC bits of the ADC14CTL0 register. After each conversion, ADC
     can be automatically or manually triggered to convert next sample.
multiSampleConvert: the iteration method to convert next sample. It can
     be one of ADC_MANUAL_ITERATION or ADC_AUTOMATIC_ITERATION.
*/

bool ADC14_disableSampleTimer(void)
/*
Enables the timer trigger for sampling by modifying ADC14SHP bit of the
     ADC14CTL0 register.
*/

void ADC14_enableInterrupt(uint_fast64_t mask)
/*
Enables the ADC interrupt by modifying the ADC14IER0 register.
mask: the interrupt type to be enabled. It can be logical OR form of
    ADC_INT0 through ADC_INT31, ADC_IN_INT, ADC_LO_INT, ADC_HI_INT,
    ADC_OV_INT, ADC_TOV_INT or ADC_RDY_INT.
*/

void ADC14_disableInterrupt(uint_fast64_t mask)
/*
Disables the ADC interrupts by modifying the ADC14IER0 register.
mask: the interrupt type to be disabled. It can be logical Or form of
    ADC_INT0 through ADC_INT31, ADC_IN_INT, ADC_LO_INT, ADC_HI_INT,
    ADC_OV_INT, ADC_TOV_INT or ADC_RDY_INT values.
*/

uint_fast64_t ADC14_getEnabledInterruptStatus(void)
/*
```

© McGraw-Hill Education. All rights reserved. Any use is subject to the Terms of Use, Privacy Notice and copyright information.

```
Returns the status of enabled interrupt flags by reading the ADC14IER0
    and ADC14IFGR0 registers.
*/

void ADC14_clearInterruptFlag(uint_fast64_t mask)
/*
Clears ADC interrupts by modifying the ADC14CLRIFGR0 register.
mask: the interrupt type to be cleared. It can be logical OR form of
    ADC_INT0 through ADC_INT31, ADC_IN_INT, ADC_LO_INT, ADC_HI_INT,
    ADC_OV_INT, ADC_TOV_INT or ADC_RDY_INT.
*/
```

8.4.6. ADC14 Module Usage via Energia Function

Energia can also be used to control the ADC14 module in MSP432. We provide the related function below. More information on Energia functions can be found in [12].

```
int analogRead(pin);
/*
Configures and reads analog value from specified pin by modifying and
    reading ADC14CTL0, ADC14CTL1, ADC14MCTLx, and ADC14MEMx registers.
pin: selected pin to be configured as analog input.
*/
```

8.4.7. Coding Practices for the ADC14 Module

We provide example codes on the usage of the ADC14 module in this section. We consider register-level, DriverLib, and Energia function usages through these examples. We use the input channels A0 (connected to pin P5.5) and A1 (connected to pin P5.4) on the MSP432 LaunchPad in some operations. We also use LED1 connected to pin P1.0 on the MSP432 LaunchPad as output.

8.4.7.1. Register-Level Examples

The first example on the usage of the ADC14 module is actually on forming a comparator. We provide the corresponding C code in Listing 8.8. Here, the input voltage level at channel A0 is checked in an infinite loop. If the converted value is above a certain level, LED1 is turned on. We used single-channel, single-conversion mode in this example using software trigger.

© McGraw-Hill Education. All rights reserved. Any use is subject to the Terms of Use, Privacy Notice and copyright information.

Listing 8.8 Usage of the ADC14 Module as a Comparator, Using Registers

```
#include "msp.h"

#define LED1 BIT0

void main(void){

 WDT_A->CTL = WDT_A_CTL_PW | WDT_A_CTL_HOLD;

 P1->DIR = LED1;
 P5->SEL1 |= BIT5; // Set P5.5 as ADC input A0
 P5->SEL0 |= BIT5; // Set P5.5 as ADC input A0

// Configure MCLK as 24 MHz from DCO
 CS->KEY = CS_KEY_VAL;
 CS->CTL0 &= ~ CS_CTL0_DCORSEL_MASK;
 CS->CTL0 |= CS_CTL0_DCORSEL_4;
 CS->CTL1 &= ~(CS_CTL1_SELM_MASK | CS_CTL1_DIVM_MASK);
 CS->CTL1 |= CS_CTL1_SELM_3;
 CS->KEY = 0;

 ADC14->CTL0 &= ~ADC14_CTL0_ENC; // Reset ENC bit for configuration
// Predivide ADC clock by 4, Software trigger, Sampling timer is the
    source of sampling signal, Divide ADC clock by 6, ADC clock source
    is MCLK, single channel-single conversion mode, Sample and hold
    time is 16 ADC clock cycles, ADC14 on
 ADC14->CTL0 = ADC14_CTL0_PDIV_1 | ADC14_CTL0_SHS_0 | ADC14_CTL0_SHP |
     ADC14_CTL0_DIV_5 | ADC14_CTL0_SSEL_3 | ADC14_CTL0_CONSEQ_0 |
     ADC14_CTL0_SHT0_2 | ADC14_CTL0_ON;
 ADC14->CTL1 = ADC14_CTL1_RES_3; // ADC 14-bit resolution
 ADC14->MCTL[0] = ADC14_MCTLN_VRSEL_0 | ADC14_MCTLN_INCH_0;  // Vcc -
     Vss references, input channel A0 is selected
 ADC14->CTL0 |= ADC14_CTL0_ENC; // Enable conversion

 while (1){
  ADC14->CTL0 |= ADC14_CTL0_SC; // Start conversion
  while(ADC14->CTL0 & ADC14_CTL0_BUSY); // Wait until conversion is
      done
  if(ADC14->MEM[0] > 0x200) P1->OUT |= LED1;
  else P1->OUT &= ~LED1;}
}
```

The second example on the usage of the ADC14 module is on measuring the internal temperature level of the MSP432 microcontroller. We provide the corresponding C code in Listing 8.9. Here, the internal temperature sensor is used with 2.5-V internal reference. We use single-channel, multiple conversion mode in this example using software trigger. The system is put in LPM0_LDO_VCORE0 mode. Hence, it waits for an ADC14 interrupt. Then, the temperature value is read and converted to degrees Celsius in the ISR.

Listing 8.9 Measuring the Internal Temperature Level of the Microcontroller by the ADC14 Module, Using Registers

© McGraw-Hill Education. All rights reserved. Any use is subject to the Terms of Use, Privacy Notice and copyright information.

```
#include "msp.h"

int TempADC;
float TempC;

int TempCal_2_5v_30C; // Variable to store the calibration value for
    2.5V reference and 30 Celsius
int TempCal_2_5v_85C; // Variable to store the calibration value for
    2.5V reference and 85 Celsius

void main(void){

 WDT_A->CTL = WDT_A_CTL_PW | WDT_A_CTL_HOLD;

// Read the ADC temperature reference calibration value
 TempCal_2_5v_30C = TLV->ADC14_REF2P5V_TS30C;
 TempCal_2_5v_85C = TLV->ADC14_REF2P5V_TS85C;

 while(REF_A->CTL0 & REF_A_CTL0_GENBUSY); // Wait until REFGENBUSY bit
     is reset
 REF_A->CTL0 |= REF_A_CTL0_VSEL_3 | REF_A_CTL0_ON; // Set reference
     voltage to 2.5 V, enable reference
 REF_A->CTL0 &= ~REF_A_CTL0_TCOFF; // Enable temperature sensor
 while(!(REF_A->CTL0 & REF_A_CTL0_GENRDY)); // Wait until reference
     voltage output is ready

 ADC14->CTL0 &= ~ADC14_CTL0_ENC; // Reset ENC bit for configuration
//Software trigger, Sampling timer is the source of sampling signal,
    single channel-repeated conversion mode, Sample and hold time is
    128 ADC clock cycles, multiple sample and conversion, ADC14 on
 ADC14->CTL0 = ADC14_CTL0_SHS_0 | ADC14_CTL0_SHP |
     ADC14_CTL0_CONSEQ_2 | ADC14_CTL0_SHT0_6 | ADC14_CTL0_MSC |
     ADC14_CTL0_ON;
 ADC14->CTL1 = ADC14_CTL1_RES_3 | ADC14_CTL1_TCMAP; // ADC 14-bit
     resolution, Enable internal temperature sensor
 ADC14->MCTL[0] = ADC14_MCTLN_VRSEL_1 | ADC14_MCTLN_INCH_22;  // VREF -
      Vss references, input channel A22 is selected
 ADC14->IER0 = ADC14_IER0_IE0; // ADC interrupt enabled for ADC14->
     MEM[0]
 ADC14->CTL0 |= ADC14_CTL0_ENC|ADC14_CTL0_SC; // Enable and start
     conversion

 NVIC->ISER[0] = 0x01000000; // ADC14 interrupt is enabled in NVIC
 __enable_irq(); // All interrupts are enabled

 SCB->SCR |= SCB_SCR_SLEEPONEXIT_Msk; // Sleep on exit
 __sleep(); // enter LPM0
}

// ADC14 interrupt service routine
void ADC14_IRQHandler(void){
 if (ADC14->IFGR0 & ADC14_IFGR0_IFG0){
  TempADC = ADC14->MEM[0]; // Read temp. sensor ADC value
// Obtain temperature in Celsius
  TempC = (((float) TempADC - TempCal_2_5v_30C) * (85 - 30)) /
      (TempCal_2_5v_85C - TempCal_2_5v_30C) + 30.0f;}
}
```

The third example on the usage of the ADC14 module focuses on the multi-channel, multiple conversion mode. We provide the corresponding C code for this operation in Listing 8.10. Here, the ADC14 module is triggered by using the Timer_A0 output 1. The microcontroller is put into LPM0_LDO_VCORE0 mode. Hence, it waits for an ADC interrupt. The conversion results of input channels A0 and A1 are obtained in the ISR every 0.5 seconds.

© McGraw-Hill Education. All rights reserved. Any use is subject to the Terms of Use, Privacy Notice and copyright information.

Listing 8.10 ADC Operation on multi-channels, Using Registers

```
#include "msp.h"

uint16_t resultsBuffer[2];

void main(void){

 WDT_A->CTL = WDT_A_CTL_PW | WDT_A_CTL_HOLD;

 P5->SEL1 |= BIT5 | BIT4; // Set P5.5 as ADC input A0 and P5.4 as ADC
     input A1
 P5->SEL0 |= BIT5 | BIT4; // Set P5.5 as ADC input A0 and P5.4 as ADC
     input A1

 CS->KEY = CS_KEY_VAL;
 CS->CTL1 |= CS_CTL1_SELA_2;
 CS->KEY = 0;

 ADC14->CTL0 &= ~ADC14_CTL0_ENC; // Reset ENC bit for configuration
//Timer_A0 output 1 trigger, Sampling timer is the source of sampling
    signal, multi channel-repeated conversion mode, Sample and hold
    time is 96 ADC clock cycles, ADC14 on
 ADC14->CTL0 = ADC14_CTL0_SHS_1 | ADC14_CTL0_SHP |
     ADC14_CTL0_CONSEQ_3 | ADC14_CTL0_SHT0_5 | ADC14_CTL0_ON;
 ADC14->MCTL[0] = ADC14_MCTLN_VRSEL_0 | ADC14_MCTLN_INCH_0; // Vcc -
     Vss references, input channel A0, End of sequence
 ADC14->MCTL[1] = ADC14_MCTLN_VRSEL_0 | ADC14_MCTLN_INCH_1 |
     ADC14_MCTLN_EOS; // Vcc - Vss references, input channel A1, End of
      sequence
 ADC14->IER0 = ADC14_IER0_IE1; // ADC interrupt enabled for ADC14->MEM
     [1]

 ADC14->CTL0 |= ADC14_CTL0_ENC; // Start conversion-software trigger

 TIMER_A0->CCR[0] = 16384;  // PWM period
 TIMER_A0->CCR[1] = 16384; // CCR1 PWM duty cycle
 TIMER_A0->CCTL[1] = TIMER_A_CCTLN_OUTMOD_3; // CCR1 set/reset
 TIMER_A0->CTL = TIMER_A_CTL_TASSEL_1 | TIMER_A_CTL_MC_1 |
     TIMER_A_CTL_CLR; // ACLK, Up mode, Clear TAR

 NVIC->ISER[0] = 0x01000000; // ADC14 interrupt is enabled in NVIC
 __enable_irq(); // All interrupts are enabled

 SCB->SCR |= SCB_SCR_SLEEPONEXIT_Msk; // Sleep on exit
 __sleep(); // enter LPM0
}

// ADC14 interrupt service routine
void ADC14_IRQHandler(void){
 if (ADC14->IFGR0 & ADC14_IFGR0_IFG1){
  resultsBuffer[0] = ADC14->MEM[0];
  resultsBuffer[1] = ADC14->MEM[1];
  __no_operation();}
}
```

8.4.7.2. DriverLib Examples

© McGraw-Hill Education. All rights reserved. Any use is subject to the Terms of Use, Privacy Notice and copyright information.

We can repeat the previous example in Listing 8.8 using DriverLib functions. We provide the C code in Listing 8.11. Here, we first set the DCO frequency as 24 MHz and LED1 as output. We set pin P5.5 as analog input using the `GPIO_setAsPeripheralModuleFunctionInputPin` function. Afterwards, we enable and initialize the ADC14 module using the `ADC14_enableModule` and `ADC14_initModule` functions. We set the ADC14 module in single sample mode using the `ADC14_configureSingleSampleMode` function. Afterwards, we configure the ADC_MEM0 as conversion memory using the `ADC14_configureConversionMemory` function. Finally, we configure the trigger as software and enable the ADC conversion using the `ADC14_enableSampleTimer` and `ADC14_enableConversion` functions. In a while loop, we trigger the ADC14 module to get a sample and wait until ADC completes the conversion using `ADC14_toggleConversionTrigger` and `ADC14_isBusy` functions. We read the conversion result using the function `ADC14_getResult` and toggle LED1 accordingly.

Listing 8.11 Usage of the ADC14 Module as a Comparator, Using DriverLib Functions

```
#include <ti/devices/msp432p4xx/driverlib/driverlib.h>

static volatile uint16_t curADCResult;

void main(void){

 WDT_A_holdTimer();

 CS_setDCOFrequency(CS_24MHZ);

 GPIO_setAsOutputPin(GPIO_PORT_P1, GPIO_PIN0);
 GPIO_setAsPeripheralModuleFunctionInputPin(GPIO_PORT_P5, GPIO_PIN5,
     GPIO_TERTIARY_MODULE_FUNCTION);

 ADC14_enableModule();
 ADC14_initModule(ADC_CLOCKSOURCE_MCLK, ADC_PREDIVIDER_1,
    ADC_DIVIDER_4, 0);
 ADC14_configureSingleSampleMode(ADC_MEM0, false);
 ADC14_configureConversionMemory(ADC_MEM0,
    ADC_VREFPOS_AVCC_VREFNEG_VSS, ADC_INPUT_A0, false);
 ADC14_enableSampleTimer(ADC_MANUAL_ITERATION);
 ADC14_enableConversion();

 while (1){
  ADC14_toggleConversionTrigger();
  while(ADC14_isBusy());
  curADCResult = ADC14_getResult(ADC_MEM0);
  if(curADCResult > 0x200)
   GPIO_setOutputHighOnPin(GPIO_PORT_P1, GPIO_PIN0);
  else
   GPIO_setOutputLowOnPin(GPIO_PORT_P1, GPIO_PIN0);}
}
```

We next consider the example in Listing 8.9 using DriverLib functions. We provide the corresponding C code in Listing 8.12. Here, we configure the ADC14 module similar to the previous example. We set the ADC input channel as temperature sensor with reference voltage as 2.5 V. Different from the previous example, we set the ADC14 module in single-channel, multiple conversion mode using the `ADC14_enableSampleTimer` function. We enable ADC interrupts using the `ADC14_enableInterrupt` function. In the ISR, we read and clear the ADC interrupt flag using `ADC14_getEnabledInterruptStatus` and `ADC14_clearInterruptFlag` functions. Then, we read the conversion result using the `ADC14_getResult` function. We finally compute the temperature value in degrees Celsius.

© McGraw-Hill Education. All rights reserved. Any use is subject to the Terms of Use, Privacy Notice and copyright information.

Listing 8.12 Measuring the Internal Temperature Level of the Microcontroller by the ADC14 Module, Using DriverLib Functions

```
#include <ti/devices/msp432p4xx/driverlib/driverlib.h>

volatile float tempC;
int cal30, cal85;

void main(void){

 WDT_A_holdTimer();

 cal30 = SysCtl_getTempCalibrationConstant(SYSCTL_2_5V_REF,
     SYSCTL_30_DEGREES_C);
 cal85 = SysCtl_getTempCalibrationConstant(SYSCTL_2_5V_REF,
     SYSCTL_85_DEGREES_C);

 REF_A_setReferenceVoltage(REF_A_VREF2_5V);
 REF_A_enableReferenceVoltage();
 REF_A_enableTempSensor();

 ADC14_enableModule();
 ADC14_initModule(ADC_CLOCKSOURCE_MCLK, ADC_PREDIVIDER_1,
     ADC_DIVIDER_1, ADC_TEMPSENSEMAP);
 ADC14_configureSingleSampleMode(ADC_MEM0, true);
 ADC14_configureConversionMemory(ADC_MEM0,
     ADC_VREFPOS_INTBUF_VREFNEG_VSS, ADC_INPUT_A22, false);
 ADC14_enableSampleTimer(ADC_AUTOMATIC_ITERATION);
 ADC14_enableInterrupt(ADC_INT0);

 Interrupt_enableSleepOnIsrExit();
 Interrupt_enableInterrupt(INT_ADC14);
 Interrupt_enableMaster();

 ADC14_enableConversion();
 ADC14_toggleConversionTrigger();
}

void ADC14_IRQHandler(void){
 uint64_t status;
 int32_t  TempADC;

 status = ADC14_getEnabledInterruptStatus();
 ADC14_clearInterruptFlag(status);

 if(status & ADC_INT0){
  TempADC = ADC14_getResult(ADC_MEM0);
  tempC = (((float)TempADC - cal30) * 55) / (cal85 - cal30) + 30.0f;}
}
```

We next consider the example in Listing 8.10 using DriverLib functions. We provide the corresponding C code in Listing 8.13. Here, we set the ADC14 module in multi-channel, multiple conversion mode. We set its trigger as Timer_A0 output 1 using `ADC14_configureMultiSequenceMode` and `ADC14_setSampleHoldTrigger` functions. In the ISR, we read and clear the interrupt flag first. Then, we read multiple channel conversion results using the `ADC14_getMultiSequenceResult` function.

Listing 8.13 ADC Operation on Multiple Channels, Using DriverLib Functions

© McGraw-Hill Education. All rights reserved. Any use is subject to the Terms of Use, Privacy Notice and copyright information.

```
#include <ti/devices/msp432p4xx/driverlib/driverlib.h>

const Timer_A_UpModeConfig upModeConfig =
{
 TIMER_A_CLOCKSOURCE_ACLK,
 TIMER_A_CLOCKSOURCE_DIVIDER_1,
 16384,
 TIMER_A_TAIE_INTERRUPT_DISABLE,
 TIMER_A_CCIE_CCR0_INTERRUPT_DISABLE,
 TIMER_A_DO_CLEAR
};

const Timer_A_CompareModeConfig compareConfig =
{
 TIMER_A_CAPTURECOMPARE_REGISTER_1,
 TIMER_A_CAPTURECOMPARE_INTERRUPT_DISABLE,
 TIMER_A_OUTPUTMODE_SET_RESET,
 16384
};

uint16_t resultsBuffer[2];

void main(void){

 WDT_A_holdTimer();

 CS_setReferenceOscillatorFrequency(CS_REFO_32KHZ);
 CS_initClockSignal(CS_ACLK, CS_REFOCLK_SELECT, CS_CLOCK_DIVIDER_1);

 Timer_A_configureUpMode(TIMER_A0_BASE, &upModeConfig);
 Timer_A_initCompare(TIMER_A0_BASE, &compareConfig);

 GPIO_setAsPeripheralModuleFunctionInputPin(GPIO_PORT_P5, GPIO_PIN4 | GPIO_PIN5,
     GPIO_TERTIARY_MODULE_FUNCTION);

 ADC14_enableModule();
 ADC14_initModule(ADC_CLOCKSOURCE_MCLK, ADC_PREDIVIDER_1, ADC_DIVIDER_1, 0);
 ADC14_configureMultiSequenceMode(ADC_MEM0, ADC_MEM1, true);
 ADC14_configureConversionMemory(ADC_MEM0, ADC_VREFPOS_AVCC_VREFNEG_VSS,
     ADC_INPUT_A0, false);
 ADC14_configureConversionMemory(ADC_MEM1, ADC_VREFPOS_AVCC_VREFNEG_VSS,
     ADC_INPUT_A1, false);

 ADC14_setSampleHoldTrigger(ADC_TRIGGER_SOURCE1, false);
 ADC14_enableSampleTimer(ADC_MANUAL_ITERATION);
 ADC14_enableInterrupt(ADC_INT1);
 ADC14_enableConversion();

 Interrupt_enableSleepOnIsrExit();
 Interrupt_enableInterrupt(INT_ADC14);
 Interrupt_enableMaster();

 Timer_A_startCounter(TIMER_A0_BASE, TIMER_A_UP_MODE);

 PCM_setPowerState(PCM_LPM0_LDO_VCORE1);
}

void ADC14_IRQHandler(void){
 uint64_t status;

 status = ADC14_getEnabledInterruptStatus();
 ADC14_clearInterruptFlag(status);

 if (status & ADC_INT1)
  ADC14_getMultiSequenceResult(resultsBuffer);
}
```

© McGraw-Hill Education. All rights reserved. Any use is subject to the Terms of Use, Privacy Notice and copyright information.

We finally consider the window comparator usage via DriverLib functions. We provide the corresponding C code in Listing 8.14. Here, we set the ADC14 module similar to Listing 8.12. We compute the threshold levels for 20°C and 50°C, respectively. Then, we set the window comparator using the function `ADC14_setComparatorWindowValue`. Afterwards, we enable the window comparator for the selected memory location using the function `ADC14_enableComparatorWindow`. Finally, we enable the ADC interrupt for the window comparator. In the ISR, we read and clear the interrupt flag first. If the result is either greater than the lower threshold level or lower than the upper threshold level, we enable the ADC conversion interrupt and turn on LED1 connected to pin P1.0 on the MSP432 LaunchPad. Then, we read the conversion result, compute the temperature value in degrees Celsius, and turn off LED1.

Listing 8.14 ADC Window Comparator Operation, Using DriverLib Functions

© McGraw-Hill Education. All rights reserved. Any use is subject to the Terms of Use, Privacy Notice and copyright information.

```
#include <ti/devices/msp432p4xx/driverlib/driverlib.h>

volatile float tempC;
int cal30, cal85, comp_h, comp_l;

void main(void){

 WDT_A_holdTimer();

 cal30 = SysCtl_getTempCalibrationConstant(SYSCTL_2_5V_REF,
     SYSCTL_30_DEGREES_C);
 cal85 = SysCtl_getTempCalibrationConstant(SYSCTL_2_5V_REF,
     SYSCTL_85_DEGREES_C);
 comp_l = ((int32_t) ((20.0 - 30.0) * (cal85 - cal30)) / 55) + cal30;
 comp_h = ((int32_t) ((50.0 - 30.0) * (cal85 - cal30)) / 55) + cal30;

 GPIO_setAsOutputPin(GPIO_PORT_P1, GPIO_PIN0);

 REF_A_setReferenceVoltage(REF_A_VREF2_5V);
 REF_A_enableReferenceVoltage();
 REF_A_enableTempSensor();

 ADC14_enableModule();
 ADC14_initModule(ADC_CLOCKSOURCE_MCLK, ADC_PREDIVIDER_1,
     ADC_DIVIDER_1, ADC_TEMPSENSEMAP);
 ADC14_configureSingleSampleMode(ADC_MEM0, true);
 ADC14_configureConversionMemory(ADC_MEM0,
     ADC_VREFPOS_INTBUF_VREFNEG_VSS, ADC_INPUT_A22, false);
 ADC14_setComparatorWindowValue(ADC_COMP_WINDOW0, comp_l, comp_h);
 ADC14_enableComparatorWindow(ADC_MEM0, ADC_COMP_WINDOW0);
 ADC14_enableSampleTimer(ADC_AUTOMATIC_ITERATION);
 ADC14_clearInterruptFlag(ADC_IN_INT);
 ADC14_enableInterrupt(ADC_IN_INT);

 Interrupt_enableSleepOnIsrExit();
 Interrupt_enableInterrupt(INT_ADC14);
 Interrupt_enableMaster();

 ADC14_enableConversion();
 ADC14_toggleConversionTrigger();

 PCM_setPowerState(PCM_LPM0_LDO_VCORE1);
}

void ADC14_IRQHandler(void){
 uint64_t status;
 int32_t  TempADC;

 status = ADC14_getEnabledInterruptStatus();
 ADC14_clearInterruptFlag(status);

 if(status & ADC_INT0){

  TempADC = ADC14_getResult(ADC_MEM0);
  tempC = (((float)TempADC - cal30) * 55) / (cal85 - cal30) + 30.0f;
  ADC14_disableInterrupt(ADC_INT0);
  GPIO_setOutputLowOnPin(GPIO_PORT_P1, GPIO_PIN0);}
 else if(status & ADC_IN_INT ){
  GPIO_setOutputHighOnPin(GPIO_PORT_P1, GPIO_PIN0);
  ADC14_enableInterrupt(ADC_INT0);}
}
```

© McGraw-Hill Education. All rights reserved. Any use is subject to the Terms of Use, Privacy Notice and copyright information.

8.4.7.3. Energia Example

We finally repeat the example given in Listing 8.8 using Energia functions. We provide the corresponding C code in Listing 8.15. Here, first we set LED1 as output in the `setup` function. Then, we configure the ADC14 module and read the conversion result using the `analogRead` function and toggle LED1 accordingly.

Listing 8.15 Usage of the ADC14 Module as a Comparator, Using Energia Functions

```
int analogIn = A0;
int curADCResult;

void setup(){
 pinMode(YELLOW_LED, OUTPUT);
}

void loop(){
 curADCResult = analogRead(analogIn);
 if(curADCResult > 0x200) digitalWrite(YELLOW_LED, HIGH);
 else digitalWrite(YELLOW_LED, LOW);
 delay(100);
}
```

© McGraw-Hill Education. All rights reserved. Any use is subject to the Terms of Use, Privacy Notice and copyright information.